Task 4: Structure Forensics (7 Marks)

Task 4.1: Locate the Defects (2 Marks)

- **Balances unchanged (logic defect).** addInterest(struct account a, ...) takes the structure **by value**. C copies list[i] into the parameter a; the function adds interest to this private copy, and the copy is destroyed when the function returns. The original list[i] is never touched.
- **Statement that does not compile.** In printf("%d %f\n", p.number, p.balance);, p is a **pointer** (struct account *p = list;). The dot operator needs a structure, not a pointer, so the compiler reports "'p' is a pointer; did you mean to use '->'?". The correct forms are p->number or (*p).number.
- Additional points: the program lacks #include <stdio.h> (so printf is undeclared), and p points only to list[0], so even after the fix only the first account would be printed.

Output

```
t4_orig.c:11:24: error: 'p' is a pointer; did you mean to use '->'?
t4_orig.c:11:34: error: 'p' is a pointer; did you mean to use '->'?
```

Task 4.2: Construct Evidence (2 Marks)

Printing the address of the structure in both functions shows that they are different objects:

```c
#include <stdio.h>

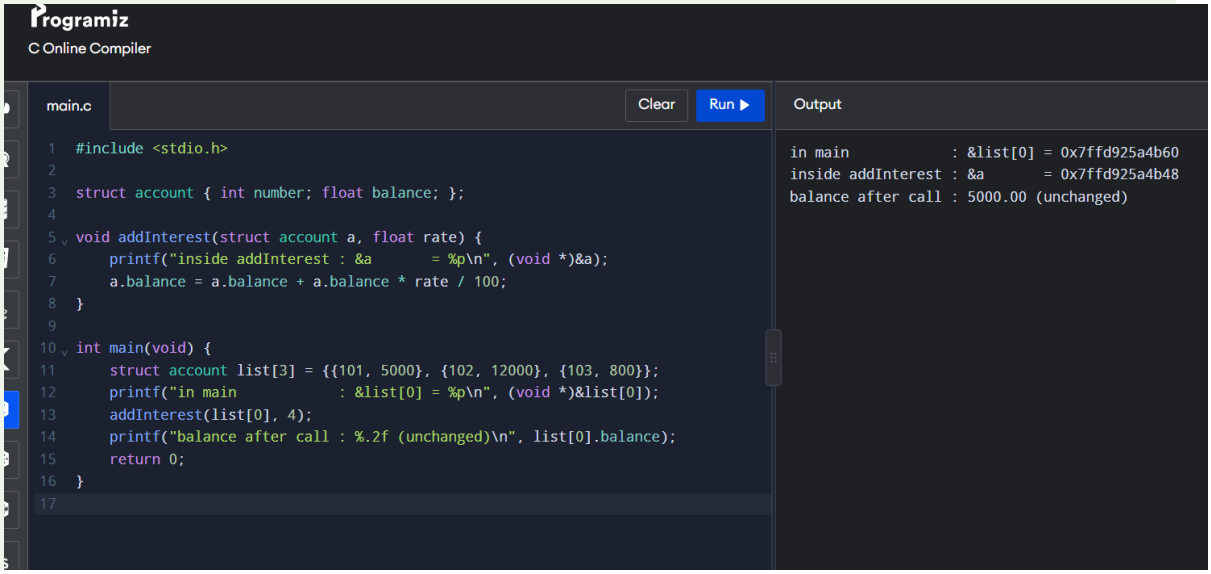
struct account { int number; float balance; };

void addInterest(struct account a, float rate) {
    printf("inside addInterest : &a       = %p\n", (void *)&a);
    a.balance = a.balance + a.balance * rate / 100;
}

int main(void) {
    struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
    printf("in main           : &list[0] = %p\n", (void *)&list[0]);
    addInterest(list[0], 4);
    printf("balance after call : %.2f (unchanged)\n", list[0].balance);
    return 0;
}
```

Output

```
in main           : &list[0] = 0x7ffd3cd46e10
inside addInterest : &a       = 0x7ffd3cd46df8
balance after call : 5000.00 (unchanged)
```



The two addresses are different, so a is a separate copy (on the called function's stack frame), and the balance in main is still 5000.00. Absolute addresses vary between runs; what matters is that they are not equal.

Task 4.3: Repair the Program (2 Marks)

```
#include <stdio.h>

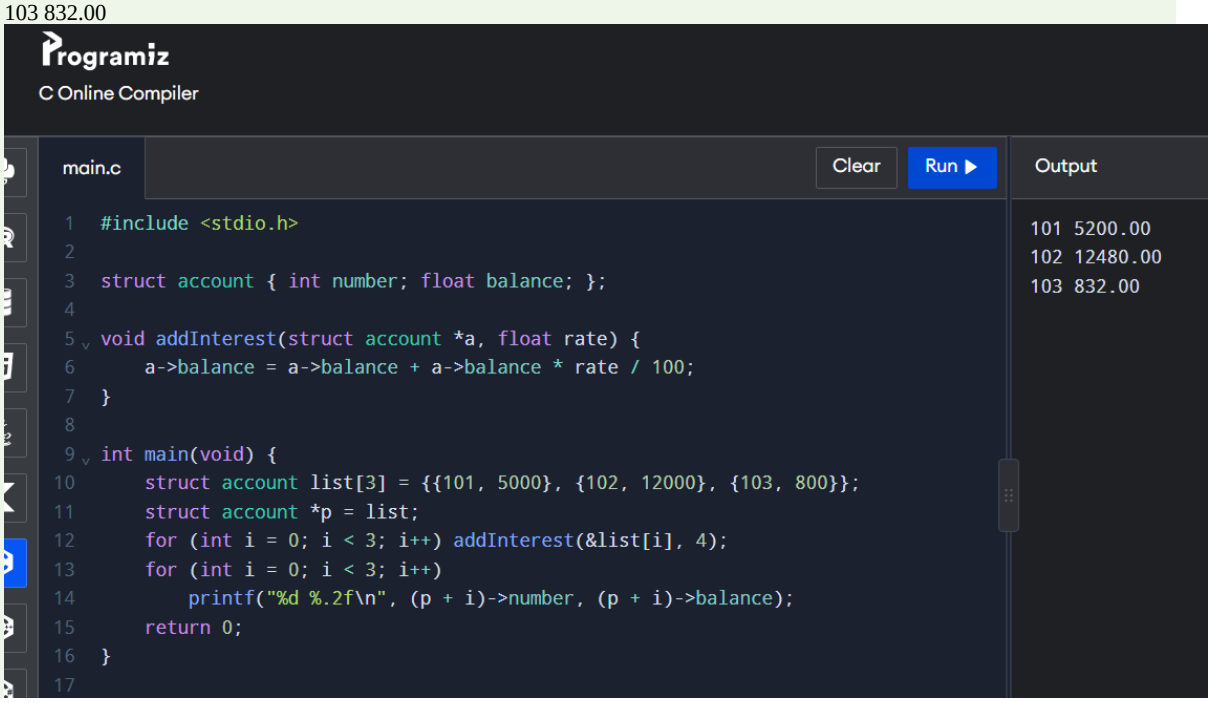
struct account { int number; float balance; };

void addInterest(struct account *a, float rate) {
   a->balance = a->balance + a->balance * rate / 100;
}

int main(void) {
   struct account list[3] = {{101, 5000}, {102, 12000}, {103, 800}};
   struct account *p = list;
   for (int i = 0; i < 3; i++) addInterest(&list[i], 4);
   for (int i = 0; i < 3; i++)
      printf("%d %.2f\n", (p + i)->number, (p + i)->balance);
   return 0;
}
```

Output

```
101 5200.00
102 12480.00
103 832.00
```



The function receives the address of the real structure (&list[i]) and the arrow operator a->balance modifies the original. The print statement uses (p + i)->number so all three accounts are shown.

Task 4.4: Prevent Recurrence (1 Mark)

Pass	Use when	Example
By value	The structure is small and the function needs its own private copy, or must not affect the caller.	double area(struct rect r)
By pointer	The function must modify the caller's structure, or the structure is large and copying is wasteful.	void addInterest(struct account *a, float rate)

Pass	Use when	Example
By const pointer	The function only reads a (usually large) structure. This avoids the copy and the compiler rejects accidental writes.	void print_parcel(const struct parcel *p)

Alternative Concept and Complexity Analysis: Return by Value

```
struct account addInterest(struct account a, float rate) {
    a.balance = a.balance + a.balance * rate / 100;
    return a;                       /* copy of the modified structure */
}
/* caller */
list[i] = addInterest(list[i], 4);
```

This works because the modified copy is returned and assigned back over the original. With sizeof(struct account) = 8 bytes (measured):

	Pass by value and return the copy	Pass a pointer
Bytes copied per call	8 (argument) + 8 (return value) = 16, plus 8 more for the assignment into list[i] if the compiler does not elide it	8 (one pointer)
Time per call	O(S) for a structure of S bytes; O(1) here because S is fixed and small. For n accounts, O(n) in total	O(1) per call, independent of S; O(n) in total
Stack space	About 2S for the parameter and the returned copy (16 bytes here), plus the frame	One pointer (8 bytes), plus the frame
Risk	Forgetting the assignment back (addInterest(list[i], 4); alone) repeats the original bug	None for this bug; the original is modified directly

For an 8-byte structure the difference is negligible (some ABIs even pass it in registers). For a 1 KB structure the by-value version would copy about 2 KB per call, so the pointer version scales better, and it also states the intent "modify in place" more clearly.